Replicating Commercial Auto-Captioning Architectures: A Comprehensive Engineering Blueprint Using Faster-Whisper

1. Architectural Analysis of Commercial Captioning Systems

The proliferation of short-form vertical video has fundamentally altered the technical and psychological requirements for automated subtitling. In the contemporary media landscape, subtitles serve dual functions: ensuring accessibility and operating as kinetic visual hooks designed to maximize viewer retention in silent autoplay environments. Commercial non-linear editing (NLE) systems and mobile-first editing platforms, specifically DaVinci Resolve and CapCut, have integrated sophisticated Artificial Intelligence (AI) pipelines to automate the generation, timing, and styling of on-screen text. To replicate these proprietary systems using open-source tools—specifically tailored for developers operating with strict hardware and financial constraints—it is critical to first deconstruct the underlying architectures and algorithmic philosophies of these commercial platforms.

1.1 The CapCut Captioning Paradigm

CapCut's architecture is explicitly optimized for short-form, mobile-first video distribution, targeting platforms such as TikTok, Instagram Reels, and YouTube Shorts. The platform employs a highly automated, cloud-assisted pipeline that prioritizes immediate viewer engagement through dynamic, visually aggressive typography rather than traditional cinematic subtitling guidelines[1]. The automation relies on a multi-stage pipeline beginning with transcription, followed by semantic chunking, and concluding with dynamic CSS-style rendering.

The CapCut auto-captioning workflow is deeply integrated with its proprietary automatic speech recognition (ASR) engines. When a user initiates the auto-caption feature, the system uploads the audio payload to cloud servers, transcribes the spoken audio, and extracts precise word-level timecodes[1]. Once the transcript is generated, CapCut segments the text into highly readable, punchy chunks. The heuristic algorithms employed aim for maximum screen legibility on small mobile devices, typically restricting captions to a maximum of two lines and approximately 14 to 22 pixels in size, calculating roughly 32 to 42 characters per line depending on the aspect ratio[2].

The visual rendering mechanism of CapCut is where its primary value proposition lies. The system applies rich styles directly to the text chunks, utilizing dynamic text highlighting. In this model, the currently spoken word is emphasized via color changes, scaling animations, or bouncing kinetic typography to retain viewer attention on muted feeds[1]. Behind the scenes,

these layouts behave similarly to cascading style sheets (CSS) text-wrapping parameters, utilizing logical equivalents to Flexbox or CSS Grid to ensure that text remains perfectly centered, dynamically wraps around safe areas, and never spills outside the frame or clashes with platform-specific user interface (UI) overlays[1]. Ultimately, the architecture favors hardcoded, "burned-in" captions that scale seamlessly across 16:9 and 9:16 aspect ratios using auto-reframing techniques, rather than relying on standard closed captioning protocols[4].

1.2 The DaVinci Resolve Subtitling Ecosystem

In contrast to CapCut's mobile-first philosophy, Blackmagic Design’s DaVinci Resolve approaches captioning from a traditional broadcast and cinematic engineering perspective. Native auto-captioning is restricted exclusively to the paid "Studio" version of the software, utilizing an internal neural engine to transcribe audio and generate standard subtitle tracks directly on the timeline[6].

The Resolve subtitling architecture relies on two distinct rendering methodologies, each presenting specific technical limitations for modern content creators. The primary method utilizes standard Subtitle Tracks. These tracks live natively on the Edit page timeline and format text globally. While they are easily exportable as sidecar files such as SubRip Text (SRT) or WebVTT (VTT), they offer severely limited animation capabilities. Formatting is applied across the entire track, making the word-by-word dynamic highlighting popularized by CapCut virtually impossible using native subtitle tracks[6].

To circumvent these limitations, advanced editors and third-party developers utilize DaVinci's Fusion compositing engine. By converting ASR output into individual Text+ nodes or Fusion macros, developers can animate captions with per-word highlighting, complex drop shadows, and kinetic typography[6]. This is the foundational logic driving sophisticated open-source extensions like auto-subs. The auto-subs project provides a local-first integration by running ASR models (such as Whisper or Faster-Whisper) in a Rust-based backend, formatting the output, and injecting custom Fusion caption macros directly into the Resolve timeline via Lua and Python scripting APIs[11]. This extension bridges the gap for free-tier users, proving that commercial-grade, dynamic captioning can be achieved natively within Resolve without purchasing the Studio license.

1.3 Strategic Blueprint for Cost-Constrained Replication

For a developer aiming to replicate these capabilities at zero financial cost, the architecture must abandon expensive cloud APIs and paid NLE licenses entirely. The optimal solution is an orchestration pipeline constructed locally using Python. This pipeline must utilize the highly efficient faster-whisper library for transcription and word-level timestamp generation. It must then apply custom, mathematically rigorous Python logic for intelligent text chunking that mimics human reading patterns.

The final visual rendering can be achieved through two distinct deployment paths depending on the user's workflow preference. The first path generates Advanced SubStation Alpha (ASS) files—a deeply programmable subtitle format that supports rich styling and per-word "karaoke" highlighting—which are then burned permanently into the video utilizing FFmpeg[14]. The second

path leverages the DaVinci Resolve Python API to programmatically generate and manipulate Text+ clips on the timeline, yielding fully editable, non-destructive dynamic captions[17].

2. The Core Transcription Engine: Faster-Whisper

To achieve transcription speeds viable for rapid video production without requiring enterprise-grade cloud compute or massive Video RAM (VRAM) allocations, the original OpenAI Whisper implementation must be discarded in favor of optimized inference engines. The faster-whisper library represents the industry standard for this task.

2.1 CTranslate2 Optimization and Hardware Efficiency

The faster-whisper library is a complete reimplementation of the Whisper architecture utilizing CTranslate2, a highly optimized C++ inference engine specifically designed for Transformer models. This transition strips away the extensive computational overhead of PyTorch, resulting in inference speeds up to four times faster than the original baseline OpenAI implementation, while identical accuracy metrics are strictly maintained[14].

For local environments with severe hardware constraints, faster-whisper supports 8-bit integer (int8) quantization. Neural networks fundamentally rely on matrix multiplications involving floating-point numbers. By aggressively quantizing the model weights from 16-bit or 32-bit floating points down to int8, the memory footprint is drastically reduced. The VRAM requirement for the massive large-v3 model drops from over 6GB down to approximately 3GB, all with a statistically negligible degradation in Word Error Rate (WER) of less than 0.2%[14]. Furthermore, if no dedicated Graphics Processing Unit (GPU) is available, the CTranslate2 engine executes highly optimized Single Instruction, Multiple Data (SIMD) operations to run int8 models efficiently on standard consumer Central Processing Units (CPUs), making it highly versatile for low-end hardware[20].

2.2 Voice Activity Detection and Batched Inference

One of the most profound architectural flaws in baseline encoder-decoder models like Whisper is the tendency to hallucinate text during periods of extended silence or non-speech background noise. The faster-whisper library actively mitigates this via integrated Voice Activity Detection (VAD), powered by the Silero VAD model[14].

By enabling the vad_filter parameter during inference, the engine pre-segments the audio, identifies temporal regions of absolute silence, and completely bypasses the decoder phase for those timestamps. This effectively eliminates hallucinatory looping and dramatically increases processing throughput on long audio files by starving the model of empty data[20].

Furthermore, faster-whisper supports semantic batching. Instead of processing 30-second sequential audio chunks one at a time, the BatchedInferencePipeline aggregates multiple VAD-detected segments and processes them simultaneously across the GPU cores. This parallelization methodology yields speeds exceeding 30 to 70 times real-time, depending on the GPU architecture, fundamentally resolving the latency bottlenecks associated with traditional Whisper implementations[20].

Inference Framework	Underlying Backend	Real-Time Factor (RTF)	VRAM Usage (Large-v3)	Key Production Features
OpenAI Whisper	PyTorch (FP16/FP32)	~8x	~6GB - 11GB	Baseline reference, highly unoptimized for production[14].
Faster-Whisper	CTranslate2 (INT8)	~35x - 72x	~3GB	VAD integration, batched inference, extreme memory efficiency[14].
WhisperX	CTranslate2 + Wav2Vec2	~70x	~4GB - 8GB	Advanced forced alignment for highly accurate word timestamps, speaker diarization[20].
Whisper.cpp	C++ / GGUF	~50x - 60x	~1.5GB - 3GB	Maximum portability across consumer hardware, Apple Silicon optimization[20].

2.3 Extraction of Word-Level Timestamps

To recreate the kinetic, dynamic word highlighting seen in commercial platforms, standard segment-level timing is entirely insufficient. Segment-level timing groups three to seven seconds of spoken text together under a single start and end timestamp. The architecture necessitates millisecond-accurate timing for every individual word spoken within the payload.
When the faster-whisper inference engine is initialized with the parameter word_timestamps=True, the model utilizes internal cross-attention patterns and dynamic time

warping algorithms to align the generated text tokens strictly to the underlying audio frames[20]. The output payload is a deeply structured Python object containing the text string, alongside the exact start time and the exact end time of every isolated spoken word[20]. This granular data matrix provides the foundational coordinate system required for rendering dynamic CSS-style CSS animations or ASS subtitle highlights. Advanced frameworks like WhisperX or stable-ts can be utilized to further refine these timestamps using external phoneme models, drastically reducing temporal drift, though faster-whisper provides highly competent word alignments natively[26].

3. The Psycholinguistics of Subtitle Segmentation

Acquiring precise word timestamps represents only the first phase of the pipeline. The raw list of timestamped words must be logically grouped into visual frames. Simply slicing the transcription output arbitrarily—such as splitting an array precisely every five words—results in disjointed, mathematically sterile captions that actively disrupt viewer comprehension and disrupt narrative flow[29]. A robust commercial captioning engine must strictly adhere to established cognitive and broadcast accessibility guidelines.

3.1 Spatial and Temporal Cognitive Constraints

The human visual cortex processes moving text under strict neuro-cognitive constraints. A professional algorithmic segmentation engine must continuously calculate and balance spatial limitations (how much screen real estate the text consumes) against temporal limitations (how long the text remains visible).

Formatting Constraint	Broadcast / Commercial Guideline	Rationale for Algorithmic Implementation
Characters Per Line	Max 42 characters (Latin alphabets)	Prevents the text from spilling over the horizontal edge of the video frame or crowding the safe areas of mobile UI overlays[29].
Lines Per Subtitle Page	Maximum 1 to 2 lines	Ensures the caption block occupies no more than 20% of the vertical screen space, maintaining unobstructed visibility of the underlying video subject[29].
Reading Speed Limits	Max 20 Characters Per Second (CPS)	Equates to roughly 160–180 words per minute (WPM). If text flashes faster than this

		calculated rate, viewer comprehension collapses[31].
Minimum On-Screen Duration	~0.83 seconds (20 frames at 24fps)	Prevents the subtitle from "flashing" on screen too rapidly for the brain's visual processing centers to register the existence of the text[29].
Maximum On-Screen Duration	Approximately 7 seconds	Prolonged static text causes visual fatigue, signals a pacing error, and suggests a potential desynchronization in the audio pipeline[31].

3.2 Syntactic and Semantic Chunking Logic

To prevent "garden path" reading errors—a phenomenon where a poorly placed line break forces the viewer to re-read the sentence to establish context—the text chunking algorithm must preserve "linguistic wholes"[29]. Subtitles are processed incrementally; viewers attempt to derive meaning before the sentence is complete. If the algorithm breaks a sentence awkwardly, it interrupts parsing[32].

An advanced segmentation algorithm evaluates the array of word-level timestamps produced by the faster-whisper model and applies cascading heuristic splitting logic based on natural language processing principles. First, the algorithm prioritizes punctuation. Hard breaks must always occur at terminal punctuation marks, including periods, question marks, and exclamation points, ensuring thoughts are fully encapsulated on a single screen[30]. Soft breaks can occur at commas or conjunctions, provided character limits allow[31].

Second, the system performs temporal pause detection. The algorithm calculates the absolute silence gap by subtracting the end time of the current word from the start time of the subsequent word. If this calculated silence exceeds a natural threshold—commonly set around 250 to 400 milliseconds—a forced line break is triggered. This temporal gap strongly indicates a natural breath, a dramatic pause, or a shift in thought by the speaker, and the subtitle must visually reflect this cadence[30].

Finally, the algorithm enforces a set of prohibited syntactical splits. The system must actively avoid breaking critical linguistic pairs across two different subtitle screens. For example, articles ("a", "an", "the") must remain on the same screen as their corresponding nouns. Prepositions cannot be separated from their objects, and proper nouns or titles must not be split across subtitle boundaries[31]. Implementing this requires a lookahead function; if the character limit forces a break, the algorithm must roll back the array index and execute the break before the

dependent preposition or article, rather than after it.

4. Visual Rasterization via Advanced SubStation Alpha (ASS)

With the word-level timing data successfully extracted and logically chunked, the pipeline must translate this numerical data into a visual rendering format. Standard SubRip Text (SRT) or WebVTT files are inadequate, as they natively lack the capacity to support complex, word-by-word highlight animation, spatial positioning, or typography modifications without relying on non-standard, player-specific HTML tags[33]. Therefore, to emulate CapCut, the system must utilize the Advanced SubStation Alpha (ASS) format.

4.1 Architecture of the ASS Protocol

Advanced SubStation Alpha is a deeply programmable, text-based subtitle scripting language historically utilized in complex anime localization. It operates via strict declarative sections. The [Script Info] section defines the resolution coordinate space (e.g., PlayResX and PlayResY), ensuring that the subtitles map perfectly to the video's aspect ratio[15]. The [V4+ Styles] section acts similarly to a CSS stylesheet. It defines reusable typography classes encompassing font family, pixel size, primary and secondary fill colors (using BGR hexadecimal formatting), border outlines, drop shadow offsets, and alignment matrices[16].

When these styles are burned into video using the FFmpeg libass library or wrapped by Python libraries like pysubs2, the resulting graphics are visually indistinguishable from native motion graphics generated manually in Premiere Pro or DaVinci Resolve[15].

4.2 Engineering Kinetic Highlights via Override Tags

To replicate the "karaoke-style" effect—where a word changes color precisely as it is spoken by the presenter—the ASS format utilizes specific override tags embedded linearly within the text payload. These tags instruct the rasterization engine to alter the rendering state on a micro-temporal scale.

ASS Override Tag	Syntax Example	Function in Dynamic Captions
Karaoke Fill (\k)	{\k33}Word	Fills the word with the primary color over a duration of 33 centiseconds, starting from the secondary base color[16].
Instant Fill (\K or \kf)	{\K50}Word	Initiates a smooth, progressive fill sweeping across the letters from left

		to right over 50 centiseconds[16].
Instant Border (\ko)	{\ko20}Word	Instantly removes the border prior to the highlight, keeping the text flat until spoken, popping the border in exactly when hit[16].
Alpha Fade (\alpha)	{\alpha&HFF\t(0,500,\alpha 0)}	Creates a complex fade-in animation by animating the transparency (alpha) layer from fully invisible (FF) to fully opaque (0) over 500 milliseconds[16].
Color Override (\c)	{\1c&H00FFFF&}	Hardcodes the primary text color (using ASS BGR hex formatting, e.g., Yellow) for a specific word, overriding the global style[16].

In a standard commercial short-form workflow, a caption block appears on screen in a muted color, typically white or light gray. As the speaker vocalizes each word, the word snaps instantly to a bright primary color, such as vivid yellow or neon green.

In ASS logic, this is achieved by defining a Style header where the PrimaryColour represents the active highlight color, and the SecondaryColour represents the inactive, un-highlighted state[16]. The orchestration script must iterate over the word timestamps, calculate the duration of each word in *centiseconds* (the temporal unit ASS uses for the \k tag, where 1 second equals 100 centiseconds), and inject these tags sequentially into the subtitle strings[39].

4.3 The FFmpeg Compositing Engine

Once the .ass file is procedurally generated, it serves as the instructional input matrix for the FFmpeg rendering engine. FFmpeg utilizes the subtitles or ass video filter to parse the ASS script and rasterize the text graphics directly onto the video frames during the encoding process[37].

Because rendering subtitles fundamentally requires modifying the actual pixel data of the video stream, a full video re-encode is mathematically mandatory; the video stream cannot simply be copied[37]. The FFmpeg pipeline must leverage high-efficiency codecs such as libx264 with a Constant Rate Factor (CRF) of roughly 18 to 23 to ensure near-lossless visual quality. Crucially, the audio streams must be passed through without any processing using the -c:a copy argument, guaranteeing the original audio track is perfectly preserved without generational

compression loss[15]. When utilizing custom typefaces, passing the fontsdir mapping parameter ensures FFmpeg can locate TTF or OTF fonts downloaded locally, circumventing severe system-level font caching errors[15].

5. The Codex Engineering Blueprint: Procedural Implementation

To successfully feed this design into an AI coding assistant (such as OpenAI Codex, GitHub Copilot, or ChatGPT) to generate functional Python code, the system must be broken down into tightly scoped, strictly defined sequential modules.

The following blueprint outlines the exact programmatic steps, logic constraints, data transformations, and library dependencies required. By feeding these instructions directly to a code generator, a developer can construct a standalone, automated, dynamic captioning engine entirely from open-source repositories.

5.1 Module 1: Environment and Preprocessing

The transcription model performs optimally when fed pristine, correctly formatted audio data. Extracting this data from the video container requires a subprocess call to FFmpeg.

- **Prompt Instruction for Codex:** Write a Python function using the subprocess module to invoke the FFmpeg CLI. The function must accept an input video file path and output a .wav audio file to a temporary directory.
- **Required FFmpeg Arguments:**
  - -y (Overwrite existing files).
  - -vn (Disable video processing entirely to save time)[43].
  - -acodec pcm_s16le (Export as an uncompressed 16-bit PCM WAV)[43].
  - -ar 16000 (Force the sample rate to exactly 16,000 Hz, which is the native frequency expected by the Whisper architecture)[28].
  - -ac 1 (Mix down stereo or surround tracks to a single mono channel for clean ASR ingestion)[43].

5.2 Module 2: ASR Inference via Faster-Whisper

This module loads the optimized AI model and generates the raw timestamp array.

- **Prompt Instruction for Codex:** Utilize the faster_whisper Python package to transcribe the generated WAV file.
- **Initialization Constraints:** Initialize the WhisperModel class. Set model_size="base" or "small" for speed on low-end hardware, or "large-v3" for maximum accuracy if VRAM permits. Set device="cuda" with a try-except fallback to "cpu". Critically, set compute_type="int8" to ensure the model comfortably fits within limited system memory[14].
- **Execution Constraints:** Call the model.transcribe() method. The following arguments are mandatory:
  - word_timestamps=True (Essential for capturing the precise start and end of every

word)[20].
    - vad_filter=True (Essential to prevent hallucination loops in silent audio zones)[20].
- **Data Normalization Structure:** Iterate through the generated segments generator object, and extract the nested words arrays. Flatten this highly nested structure into a single, simple Python list of dictionaries: [{"word": "Hello", "start": 0.50, "end": 0.85}, ...]. Strip any leading or trailing whitespace from the word strings[44].

5.3 Module 3: The Heuristic Semantic Chunker

This is the mathematical core of the application. It processes the flat array of words to create punchy, readable blocks of text, mimicking CapCut's proprietary pacing logic.

- **Prompt Instruction for Codex:** Write a chunking algorithm that iterates over the flat list of word dictionaries. Group them into a List[List[dict]], where each inner list represents one subtitle page/screen.
- **Algorithmic Rules to Enforce in Code:**

1. Initialize an empty current_page array and a pages array.
2. Iterate over each word dictionary. Calculate the duration of the word.
3. **Maximum Character Limit:** Keep a running tally of the total string length of current_page. If adding the next word causes the total character count to exceed 40 characters, force a page break. Push current_page to pages, clear it, and start a new current_page with the current word[29].
4. **Pause Detection Threshold:** Calculate the silence gap (current_word['start'] - previous_word['end']). If this gap is strictly greater than 0.4 seconds (400ms), force a page break *before* adding the current word, representing a natural breath[30].
5. **Punctuation Trigger:** Check the previous word's string. If it ends in a terminal punctuation mark (., ?, !), force a page break before the current word[30].
6. **Syntactic Lookahead (The Anti-Garden-Path Rule):** Implement a check against a hardcoded list of prepositions and articles (e.g., ["a", "an", "the", "in", "on", "at", "to"]). If the algorithm determines a break is required due to character limits, check if the *last* word in the current_page is in this list. If it is, pop it off the current_page, push the page, and prepend that article to the start of the *next* page. Never strand an article at the end of a subtitle screen[31].

5.4 Module 4: ASS Generation and Karaoke Styling

This module translates the chunked data into a formatted .ass text file using string manipulation. Alternatively, the prompt can instruct Codex to use the pysubs2 library, which provides an object-oriented API for building SSA/ASS files[35].

- **Prompt Instruction for Codex:** Write a function to generate an ASS file structure natively using string concatenation to avoid dependency bloat.
- **Header Definition:** Construct the [Script Info] defining PlayResX: 1080 and PlayResY: 1920 for vertical video.
- **Style Definition:** Define a [V4+ Styles] block. Create a default style named "HighlightStyle".

- Set FontName to a bold, modern typeface (e.g., "Montserrat" or "Arial Rounded MT Bold")[36].
- Set PrimaryColour (the highlight color) to Yellow (&H0000FFFF)[36].
- Set SecondaryColour (the base color) to White (&H00FFFFFF)[36].
- Set Outline to 4 and Shadow to 2. Set Alignment to 2 (Bottom Center) and MarginV to 150 to dodge TikTok/Reels UI buttons[15].

- **Event Construction (The Karaoke Logic):**
  - Define the [Events] block. For each chunk (subtitle page) in the pages array, identify the absolute start_time of the first word and the end_time of the last word. Format these into ASS timestamps: H:MM:SS.cs.
  - Iterate through the individual words within the chunk. For each word, calculate its duration in centiseconds: duration_cs = int((word['end'] - word['start']) * 100)[39].
  - Format the final string. To achieve the dynamic effect, prefix each word with the \k tag: {\k<duration_cs>}Word.
  - *Crucial Edge Case for Codex:* Ensure leading or trailing spaces between words are placed *outside* of the override braces, e.g., {\k35}Hello {\k20}world. If spaces are placed inside, formatting glitches occur in FFmpeg[16].

5.5 Module 5: Video Encoding and Hardsub Burn-in

The final python function invokes FFmpeg to composite the procedurally generated ASS file onto the original video container.

- **Prompt Instruction for Codex:** Write a subprocess call executing the FFmpeg subtitle burn-in command.
- **Target FFmpeg Command Structure:**
  ffmpeg -y -i input_video.mp4 -vf "ass=captions.ass" -c:v libx264 -crf 18 -preset fast -c:a copy final_output.mp4
- **Verification Rules:** Ensure the -vf "ass=captions.ass" filter uses the correct escaping for file paths[37]. Ensure -c:a copy is utilized to prevent audio re-encoding[37].

6. NLE Integration: Programmatic DaVinci Resolve Injection

While the FFmpeg burn-in pipeline outlined in Section 5 successfully automates CapCut-style workflows, professional editors often prefer to retain fully editable, non-destructive subtitles within their NLE. Since a cost-constrained developer likely utilizes the free version of DaVinci Resolve, native auto-captioning is entirely inaccessible[47].

However, the word-timestamp arrays generated by the faster-whisper Python pipeline can be forcefully integrated into the DaVinci Resolve timeline. This is achieved using DaVinci's officially supported Python API to procedurally generate and manipulate Text+ elements[17].

6.1 Bypassing API Limitations via Fusion Macros

Historically, developers attempted to use the Timeline.InsertSubtitleFromFile API command.

However, this method frequently experiences severe failures in modern Resolve builds, often ignoring targeted timecodes or returning null pointers[50]. Therefore, the system must manually instantiate individual Text+ Fusion composition tools on standard video tracks.

- **Environment Initialization:** The script must establish a connection to the Resolve host via dvr_script.scriptapp("Resolve"), access the Project Manager, and fetch the current timeline object using project.GetCurrentTimeline()[18]. Python environment variables (RESOLVE_SCRIPT_API and PYTHONPATH) must be strictly configured to locate the fusionscript.so or fusionscript.dll libraries[49].
- **Node Injection:** For each subtitle chunk generated in Module 3, the script creates an empty timeline item for the duration of the chunk. Because direct API placement of Text+ tools is constrained, the script must utilize the Timeline.InsertFusionTitleIntoTimeline("Text+") command, or rely on appending to the MediaPool and fetching the resultant clip[19].
- **Attribute Manipulation:** Once the Text+ node is present on the timeline, the script accesses the underlying Fusion tool properties. The Python script retrieves the Fusion composition utilizing GetFusionCompByIndex(1)[53]. It then locates the TextPlus tool node and forcefully overwrites the StyledText property with the string payload of the subtitle chunk[18].

6.2 Keyframing Dynamic Highlights in Resolve

To recreate the per-word highlight effect inside DaVinci Resolve without burning it via FFmpeg, the Python script must interact deeply with the Fusion keyframe engine.
Instead of generating hundreds of individual text clips, advanced architectures like auto-subs create a single, continuous Text+ Fusion macro that spans the entire video track. The script iterates through the faster-whisper word timestamps and programmatically sets keyframes on the StyledText parameter at specific frame numbers (calculated by multiplying the timestamp in seconds by the timeline's frames per second)[18]. When the playhead reaches a new word, the keyframe triggers, and the text payload updates instantly. By injecting Fusion modifiers or character-level styling attributes alongside the text string, the specific spoken word can be highlighted yellow, entirely replicating the CapCut aesthetic within a professional, free-tier NLE environment.

7. Conclusion

Replicating the proprietary, high-engagement captioning engines of platforms like CapCut and DaVinci Resolve Studio does not necessitate access to expensive cloud APIs, monthly subscriptions, or paid NLE tiers. The economic and computational barriers to entry have been dismantled by highly optimized open-source libraries.
By systematically deploying the CTranslate2-optimized faster-whisper model, developers can extract highly accurate, millisecond-precise text matrices even on severely restricted consumer hardware by utilizing int8 quantization and batched VAD pipelines. When this data is subjected to programmatic, linguistically aware text segmentation, the output mimics human pacing and readability. Finally, translating these data structures into Advanced SubStation Alpha (ASS)

karaoke tags and rendering through FFmpeg achieves professional-grade kinetic typography. By feeding the specific, modular architectural blueprint detailed in this report to an AI coding assistant, an engineering team can rapidly generate, deploy, and refine a zero-cost, fully automated, local-first video captioning pipeline.

Works cited

1. Seedance 2.0 For Caption Generation: A Practical CapCut Guide, https://www.capcut.com/ideas/seedance-2-0-for-caption-generation
2. Video Caption Generator: Auto-Create Subtitles - CapCut, https://www.capcut.com/resource/video-caption-generator
3. CSS Text Wrap - Explore Concept, Methods, and An Alternative Process - CapCut, https://www.capcut.com/resource/css-text-wrap
4. 2025 Recap Video: How to Make Viral Memories with AI for Free - CapCut, https://www.capcut.com/resource/recap-video
5. 9 Reliable Tools to Quickly Convert YouTube Video to Transcripts - CapCut, https://www.capcut.com/resource/youtube-videos-to-transcripts
6. How to add subtitles in DaVinci Resolve - Storyblocks, https://www.storyblocks.com/resources/tutorials/how-to-add-subtitles-davinci-resolve
7. Step-by-Step: DaVinci Resolve Auto Caption & Subtitle Guide - FireCut, https://firecut.ai/blog/step-by-step-davinci-resolve-auto-caption-subtitle-guide/
8. Can Da Vinci automatically add subtitles with just audio, or do you have to type them out manually? : r/davinciresolve - Reddit, https://www.reddit.com/r/davinciresolve/comments/1kkhuph/can_da_vinci_automatically_add_subtitles_with/
9. How to Create Subtitles in DaVinci Resolve (Import SRT, Fix Timing, Export) | GoTranscript, https://gotranscript.com/en/blog/create-subtitles-davinci-resolve-import-srt-timing-export
10. View topic - SRT Captions Formatting - Background Size / Placement Issue - Blackmagic Forum, https://forum.blackmagicdesign.com/viewtopic.php?f=38&t=200748
11. https://github.com/tmoroney/auto-subs | Ecosyste.ms: Awesome, https://awesome.ecosyste.ms/projects/github.com%2Ftmoroney%2Fauto-subs
12. Releases · tmoroney/auto-subs - GitHub, https://github.com/tmoroney/auto-subs/releases
13. GitHub - tmoroney/auto-subs: On-device subtitle generation that connects directly to DaVinci Resolve, Premiere, and After Effects., https://github.com/tmoroney/auto-subs
14. Self-Host Faster-Whisper on GPU Cloud: Production Deployment Guide for Real-Time ASR (2026) - Spheron, https://www.spheron.network/blog/faster-whisper-gpu-cloud-production-deployment-guide/
15. GitHub - nikhil-reddy05/auto-captions: Automatically generate accurate, per-word

video captions with timestamps using Whisper ASR and FFmpeg, perfect for YouTube, social media, and accessibility., https://github.com/nikhil-reddy05/auto-captions
16. ASS Override Tags - Aegisub, https://aegisub.org/docs/latest/ass_tags/
17. ResolveCafe/docs/developers/scripting.md at main - GitHub, https://github.com/CommandPost/ResolveCafe/blob/main/docs/developers/scripting.md
18. View topic - Use Python to Overlay Clip Names with Fusion in Resolve - Blackmagic Forum, https://forum.blackmagicdesign.com/viewtopic.php?f=38&t=202933
19. How to insert Text+ clips on specific video tracks using DaVinci Resolve Python API?, https://stackoverflow.com/questions/79638044/how-to-insert-text-clips-on-specific-video-tracks-using-davinci-resolve-python
20. Faster-Whisper Setup Guide (2026): 4x Faster Local Speech-to-Text | Local AI Master, https://localaimaster.com/blog/faster-whisper-guide
21. faster-whisper - PyPI, https://pypi.org/project/faster-whisper/0.2.0/
22. Faster-Whisper Guide: Faster Speech-to-Text with CTranslate2 - SayToWords, https://www.saytowords.com/blogs/Faster-Whisper-Guide/
23. (PDF) Clip Cannon White Paper - ResearchGate, https://www.researchgate.net/publication/403379216_Clip_Cannon_White_Paper
24. Deploy Whisper v4 and Production ASR on GPU Cloud: Self-Host Speech Recognition for Voice Agents, Meetings, and Call Centers (2026 Guide) | Spheron Blog, https://www.spheron.network/blog/whisper-v4-asr-gpu-cloud-production-guide/
25. Speeding up Whisper - Mobius Labs, https://mobiusml.github.io/batched_whisper_blog/
26. WhisperX with Diarization | Guides - Clore.ai, https://docs.clore.ai/guides/audio-and-voice/whisperx
27. rpayanm/stable-ts - Hugging Face, https://huggingface.co/rpayanm/stable-ts
28. WhisperX: Fast Whisper Transcription With Word Timestamps | GoTranscript, https://gotranscript.com/public/whisperx-fast-whisper-transcription-with-word-timestamps
29. Towards Automatic Subtitling: Assessing the Quality of Old and New Resources, https://journals.openedition.org/ijcol/649
30. Splitting text into subtitle pages is harder than it looks - Ryan Welch, https://ryanwelch.co.uk/blog/splitting-text-into-subtitle-pages/
31. English Closed Captioning Style Guide - Fluen AI, https://fluen.ai/post/english-closed-captioning-style-guide
32. Line breaks in subtitling: an eye tracking study on viewer preferences - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC7733619/
33. How to make words in ASS subtitles appear one at a time? - Stack Overflow, https://stackoverflow.com/questions/60608765/how-to-make-words-in-ass-subtitles-appear-one-at-a-time
34. SRT vs VTT vs TXT vs ASS: Subtitle Formats Explained - Subvideo.ai,

https://subvideo.ai/blog/subtitle-formats-srt-vtt-txt-ass/
35. API tutorial: Let's import pysubs2 - Read the Docs, https://pysubs2.readthedocs.io/en/latest/tutorial.html
36. SubTextHighlight - PyPI, https://pypi.org/project/SubTextHighlight/3.0/
37. FFmpeg Subtitles Filter: How to Burn SRT Subtitles Into Video - DEV Community, https://dev.to/javidjamae/ffmpeg-subtitles-filter-how-to-burn-srt-subtitles-into-video-2e45
38. Using pysubs2 from the Command Line - Read the Docs, https://pysubs2.readthedocs.io/en/latest/cli.html
39. Making the karaoke, https://docs.karaokes.moe/contrib-guide/create-karaoke/karaoke/index.html
40. Tutorial 1 - Aegisub 手册, https://aegi.vmoe.info/docs/3.1/Automation/Karaoke_Templater/Tutorial_1/
41. How to Add Subtitles to Video with FFmpeg, https://www.ffmpeg-micro.com/blog/ffmpeg-add-subtitles-to-video
42. HowToBurnSubtitlesIntoVideo – FFmpeg, https://trac.ffmpeg.org/wiki/HowToBurnSubtitlesIntoVideo
43. Video Processing with FFmpeg: Powering Subtilo's Subtitle Integration | by Oluwagbemiga Awosope | Medium, https://medium.com/@oluwagbemiga.awosope123/video-processing-with-ffmpeg-powering-subtilos-subtitle-integration-fe8360e6625a
44. Whisper: A local pipeline for offline subtitles (Part 1) - Hotovo, https://www.hotovo.com/blog/whisper-a-local-pipeline-for-offline-subtitles-part-1
45. API Reference — pysubs2 1.8.1 documentation, https://pysubs2.readthedocs.io/en/latest/api-reference.html
46. kalterBebapKacke/SubTextHighlight - GitHub, https://github.com/kalterBebapKacke/SubTextHighlight
47. 19.1 - Broken Scripts Megathread : r/davinciresolve - Reddit, https://www.reddit.com/r/davinciresolve/comments/1gpo20i/191_broken_scripts_megathread/
48. Converting Text+ to Subtitle track or .srt file - Blackmagic Forum • View topic, https://forum.blackmagicdesign.com/viewtopic.php?t=221552&p=1147941
49. Scripting API | DaVinci Resolve Wiki, https://wiki.dvresolve.com/developer-docs/scripting-api
50. View topic - Scripting API request: Insert subtitles at specific timecode - Blackmagic Forum, https://forum.blackmagicdesign.com/viewtopic.php?f=21&t=236935
51. DaVinci Resolve Scripting API Doc v21.0 - Gist - GitHub, https://gist.github.com/X-Raym/2f2bf453fc481b9cca624d7ca0e19de8
52. how to script adding a Title to a timeline at a particular timecode? : r/davinciresolve - Reddit, https://www.reddit.com/r/davinciresolve/comments/1ledft9/how_to_script_adding_a_title_to_a_timeline_at_a/
53. Basic Resolve API — ResolveDevDoc 0.1 documentation - Read the Docs, https://resolvedevdoc.readthedocs.io/en/latest/API_basic.html

54. Changing Text in Title Objects via DaVinci Resolve API : r/davinciresolve - Reddit, https://www.reddit.com/r/davinciresolve/comments/1dtor08/changing_text_in_title_objects_via_davinci/
55. Change attributes of multiple Text+ clips - Blackmagic Forum • View topic, https://forum.blackmagicdesign.com/viewtopic.php?f=21&t=77197